Notes on Raspberry PI

Notes about how to create RPI Linux platformswith Yocto and use the RPI.

1 Getting the resources and the tools

1.1 The Yocto recipes

In order to clone the repositories needed to build images, including QT and Open-Embedded. Open-Embedded has support for additional servers and protocols (FTP, etc.).

`20190823`: The Warrior branch of Yocto was used.

```
git clone -b warrior git://git.yoctoproject.org/poky.git

git clone -b warrior git://git.openembedded.org/meta-openembedded

git clone -b warrior git://git.yoctoproject.org/meta-raspberrypi

git clone -b warrior https://github.com/meta-qt5/meta-qt5.git
```

1.1.1 Update for Dunfell

Steps executed on 20201011 to create an image including: Yocto, Docker, Mender. The current Yocto version is called Dunfell.

Building the core-image-base.

```
git clone --depth 1 -b dunfell-23.0.3 git://git.yoctoproject.org/poky.git
git clone --depth 1 -b dunfell git://git.openembedded.org/meta-openembedded
git clone --depth 1 -b dunfell git://git.yoctoproject.org/meta-raspberrypi
```

To build the core-image-base:

```
source poky/oe-init-build-env qv_build
bitbake -c cleansstate core-image-base
bitbake core-image-base
```

Note that with Dunfell, the Yocto build for Raspberry PI does not create the rpi-sd-image file, instead, a WIC file is created.
To copy the WIC file to the sdcard, the bmaptool can be used. See: https://github.com/agherzan/meta-raspberrypi/issues/637
To install it, the command is :

```
sudo apt install bmap-tools
```

Example of the command to copy the image to the SD:

```
sudo bmaptool copy core-image-base-raspberrypi4.wic.bz2 /dev/sdb
```

1.1.2 Full command line built using Dunfell

```
cd /home/minguzzi/repo/rpi_platforms
git clone --depth 1 -b dunfell-23.0.3 git://git.yoctoproject.org/poky.git
git clone --depth 1 -b dunfell git://git.openembedded.org/meta-openembedded
git clone --depth 1 -b dunfell git://git.yoctoproject.org/meta-raspberrypi
git clone --depth 1 -b dunfell git://git.yoctoproject.org/meta-virtualization
```

```
bitbake core-image-full-cmdline --runall fetch # fetch all packages before building them
bitbake core-image-base # build the image
```

Once the image is built, to copy it to the SD card:

```
/home/minguzzi/rpi_platform/rpi4_virt/build_output/deploy/images/raspberrypi4
# In Dunfell, the image has format wic.bz2.  so the bmaptool must be used.
sudo bmaptool copy core-image-base-raspberrypi4.wic.bz2 /dev/sdb
```

To create the SDK

```
bitbake -c populate_sdk  core-image-full-cmdline
```

Below the LOCAL.CONF:. Notes:

- Using System D, I was not able to configure the ETH0, so I switched back to Sys V.

```
# Machine Selection
MACHINE = "raspberrypi4"

# Where to place downloads
DL_DIR = "/home/minguzzi/rpi_platform/downloads"

# Where to place shared-state files
SSTATE_DIR = "/home/minguzzi/rpi_platform/sstate-cache"
```

```
# Where to place the build output
TMPDIR = "/home/minguzzi/rpi_platform/rpi4_virt/build_output"

DISTRO ?= "poky"

# Package Management configuration
PACKAGE_CLASSES= "package_deb"

# SDK target architecture
SDKMACHINE="x86_64"

# Extra image configuration defaults
EXTRA_IMAGE_FEATURES = " tools-debug debug-tweaks "

# Additional image features
USER_CLASSES ?= "buildstats image-mklibs image-prelink"

# Interactive shell configuration
PATCHRESOLVE = "noop"

# Disk Space Monitoring during the build
BB_DISKMON_DIRS ??= "\
    STOPTASKS,${TMPDIR},1G,100K \
    STOPTASKS,${DL_DIR},1G,100K \
    STOPTASKS,${SSTATE_DIR},1G,100K \
    STOPTASKS,/tmp,100M,100K \
    ABORT,${TMPDIR},100M,1K \
    ABORT,${DL_DIR},100M,1K \
    ABORT,${SSTATE_DIR},100M,1K \
    ABORT,/tmp,10M,1K"

CONF_VERSION = "1"

# BB_NO_NETWORK="1"

# The number of threads that BitBake runs in parallel
BB_NUMBER_THREADS="4"

IMAGE_INSTALL_append = " packagegroup-core-full-cmdline "

IMAGE_INSTALL_append = " kbd-keymaps "

# Pro-wifi
IMAGE_INSTALL_append = " linux-firmware wpa-supplicant rpio rpi-gpio "

# Removes the splash screen on boot
IMAGE_FEATURES_remove = "splash"
DISABLE_SPLASH = "1"

# SDK
# Support for the SDK: the ssh-server-openssh substitutes the Dropbear SSH
IMAGE_FEATURES_remove = "ssh-server-dropbear"
IMAGE_FEATURES_append = " ssh-server-openssh"

# Allows the toolchain to create statically linked binaries.
IMAGE_INSTALL_append = " glibc-staticdev"

# Adds the full command line.
IMAGE_INSTALL_append = " packagegroup-core-full-cmdline"

IMAGE_INSTALL_append =" kernel-modules udev-extraconf "

# Adds support for italian keyboard.
IMAGE_LINGUAS_append = " it-it"

EXTRA_IMAGE_FEATURES += "package-management "

# 20201031 Adds FFMPEG
IMAGE_INSTALL_append =" ffmpeg"
LICENSE_FLAGS_WHITELIST="commercial"

# 20201031 Adds NFS
CORE_IMAGE_EXTRA_INSTALL += " nfs-utils "
IMAGE_INSTALL_append += " nfs-utils "

DISTRO_FEATURES_append += " bluez5 bluetooth i2c-tools wifi "
IMAGE_INSTALL_append += " bluez5 "

ENABLE_UART = "1"
VIDEO_CAMERA = "1"

# Adds support for the go language
IMAGE_INSTALL_append = " go go-dep go-helloworld "

# 8388608 = 8GB
IMAGE_ROOTFS_EXTRA_SPACE = "8388608"
NOHDD="1"
NOISO="1"

IMAGE_INSTALL_append = " linux-firmware-rpidistro-bcm43455 bluez-firmware-rpidistro-bcm4345c0-hcd  "
IMAGE_INSTALL_append += " i2c-tools "

# Configures the WPA supplicant.
IMAGE_INSTALL_append = " rbp-network-configuration "
```

2 Yocto Bitbake

2.1 The LOCAL.CONF file

Example of `local.conf` file.

```
# Machine Selection
MACHINE = "raspberrypi4"

# Where to place downloads
DL_DIR = "/home/minguzzi/rpi4_qt_platform/downloads"

# Where to place shared-state files
SSTATE_DIR = "/home/minguzzi/rpi4_qt_platform/sstate-cache"

# Where to place the build output
TMPDIR = "/home/minguzzi/rpi4_qt_platform/build/build_output"

# Default policy config
DISTRO ?= "poky"

# Package Management configuration
PACKAGE_CLASSES ?= "package_deb "

# SDK target architecture
SDKMACHINE="x86_64"

# Extra image configuration defaults
EXTRA_IMAGE_FEATURES = " tools-debug debug-tweaks "

# Interactive shell configuration
PATCHRESOLVE = "noop"

# Disk Space Monitoring during the build
BB_DISKMON_DIRS ??= "\
    STOPTASKS,${TMPDIR},1G,100K \
    STOPTASKS,${DL_DIR},1G,100K \
    STOPTASKS,${SSTATE_DIR},1G,100K \
    STOPTASKS,/tmp,100M,100K \
    ABORT,${TMPDIR},100M,1K \
    ABORT,${DL_DIR},100M,1K \
    ABORT,${SSTATE_DIR},100M,1K \
    ABORT,/tmp,10M,1K"

# CONF_VERSION is increased each time build/conf/ changes incompatibly.
CONF_VERSION = "1"


# BB_NO_NETWORK is files were already downloaded and no network access is allowed.
# BB_NO_NETWORK="1"

# The number of threads that BitBake runs in parallel
BB_NUMBER_THREADS="8"

IMAGE_INSTALL_append = " packagegroup-core-full-cmdline "

IMAGE_INSTALL_append = " kbd-keymaps "

# Pro-wifi
IMAGE_INSTALL_append = " linux-firmware wpa-supplicant linux-firmware-bcm43430 rpio rpi-gpio "

# Removes the splash screen on boot
IMAGE_FEATURES_remove = "splash"

# Support for the SDK: the ssh-server-openssh substitutes the Dropbear SSH
IMAGE_FEATURES_remove = "ssh-server-dropbear"
IMAGE_FEATURES_append = " ssh-server-openssh"

# Allows the toolchain to create statically linked binaries.
IMAGE_INSTALL_append = " glibc-staticdev"

# Adds support for Boost C++ libraries 20170924
IMAGE_INSTALL_append = " boost "

# Adds support for Bluetooth 20170924
DISTRO_FEATURES_append += " bluez5 bluetooth i2c-tools wifi "
IMAGE_INSTALL_append += " bluez5 "

# Added on 20190819 otherwise the bluetooth does not start.
DISTRO_FEATURES_append += " systemd "
VIRTUAL-RUNTIME_init_manager = "systemd"

# Adds the full command line.
IMAGE_INSTALL_append = " packagegroup-core-full-cmdline"

# GPU_MEM = "16"

IMAGE_INSTALL_append =" kernel-modules udev-extraconf "

CORE_IMAGE_EXTRA_INSTALL += " dhcp-server nfs-utils "
```

```
# Adds recipes from the meta-rbplatform layer (Raspberry Pi Platform RPB).
# Network configuration: eth0 and Wifi
# IMAGE_INSTALL_append = " rbp-network-configuration "
IMAGE_INSTALL_append = " rbp-systemd-network-configuration "

# Sets the .profile for the root on the target.
IMAGE_INSTALL_append = " rbp-root-profile "

# DHCP server configuration.
IMAGE_INSTALL_append = " rbp-dhcp-server-configuration "

# Adds support for italian. 20171005 ---------------------------
IMAGE_LINGUAS_append = " it-it"

# Adds the Web server Monkey
# IMAGE_INSTALL_append = " monkey "

EXTRA_IMAGE_FEATURES += "package-management "

IMAGE_INSTALL_append = " \
    qtbase \
    qtbase-dev \
    qtbase-mkspecs \
    qtbase-plugins \
    qtbase-tools \
    qtconnectivity-dev \
    qtconnectivity-mkspecs \
    python-smbus \
    bridge-utils \
    hostapd \
    iptables \
"
```

2.2 How-to build the image and the SDK

Note: `rpi-basic-image` is now deprecated. Use `core-image-base`

- `bitbake core-image-base` # Builds the SD image
- `bitbake -c populate_sdk core-image-base` # Builds the SDK

2.3 How to copy the resulting image into the SD card

The image file can be found, for example at the path `<BUILD>/build_output/deploy/images/raspberry4`

Example, in case the SD device is `/dev/sdb`:

```
sudo dd if=core-image-base-raspberrypi4-20190823164414.rootfs.rpi-sdimg of=/dev/sdb bs=4MB
```

Remove the existing partitions in the SD card, before copying the image.

IF=Input File, OF=Output File, BS is the block size

Additional partitions can be added to the SD, after the image is copied.

20190823: additional partitions are automatically detected and mounted by Yocto Linux at boot.

3 How-to(s)

3.1 How to configure the network interfaces (SystemD)

ETH0. Create a file named: `/etc/systemd/network/eth0.network`

```
[Match]
Name=eth0

[Network]
Address=169.254.1.2/24
Gateway=169.254.1.1
```

WLAN0. Create a file named: `/etc/systemd/network/wlan0.network`

```
[Match]
Name=wlan0

[Network]
```

```
DHCP=yes
```

WPA-Supplicant: Create a file named: `/etc/wpa_supplicant/wpa_supplicant-wlan0.conf`

Note that the name of the file is: `wpa_supplicant-`*`interface`*`.conf`

```
ctrl_interface=/var/run/wpa_supplicant
ctrl_interface_group=0
update_config=1

network={
    key_mgmt=WPA-PSK
    ssid="BBBELL-D3AD"
    psk="CsGeviP1995_BrqPX8"
}
```

Enable the WPA Supplicant on the interface wlan0. (See the link on the WPA supplicant) The command is:

```
systemctl enable wpa_supplicant@wlan0.service
```

Reboot to apply the configuration.

3.2 How to configure the network interfaces (System-V)

Modify the `/etc/network/interfaces` file as below. ETH0 is given the static address `192.168.10.5`.

```
# /etc/network/interfaces -- configuration file for ifup(8), ifdown(8)
# Wired or wireless interfaces
auto eth0
iface eth0 inet static
address 192.168.10.5
netmask 255.255.255.0

# The loopback interface
auto lo
iface lo inet loopback

# Bluetooth networking
iface bnep0 inet dhcp
```

NOTE: do not add the gateway line to the `eth0` configuration, otherwise the Raspberry will not be able to resolve names using the DNS, that is accessed by means of the wifi network.

3.3 How to remove the splash-screen at boot

In the `local.conf`, add the line:

```
IMAGE_FEATURES_remove = "splash"
```

3.4 How to add support for the Italian keyboard

The solution is described here. The following line must be added to local.conf:

```
IMAGE_INSTALL_append = " kbd-keymaps"
```

If the italian keyboard is not loaded by the `.profile` or `.bash_rc`, on the console command line, issue the command:

```
# loadkeys it
```

3.5 How to add support for WiFi

In the `local.conf` , add the line:

```
IMAGE_INSTALL_append = " linux-firmware wpa-supplicant"
```

3.5.1 System-V

Create a `wpa_supplicant.conf` file under /etc, with the following content:

```
ctrl_interface=/var/run/wpa_supplicant

ctrl_interface_group=0

update_config=1


network={

    key_mgmt=WPA-PSK

    ssid="My_Wifi_SSID"
```

```
    psk="My_Wifi_Password"
}
```

Note that the password is stored in plain text.

It is possible to create a file using the `wpa_passphrase` tool (if it is installed). A basic configuration file can be created by issuing the command (see link):

```
# wpa_passphrase MYSSID passphrase > /etc/wpa_supplicant/example.conf
```

To configure the Wlan0 with the WPA_Supplicant, add the following section to the `/etc/network/interfaces` file:

```
# Wireless interfaces
auto wlan0
iface wlan0 inet dhcp
  wireless_mode managed
  wireless_essid any
  wpa-driver wext
  wpa-conf /etc/wpa_supplicant.conf
```

Note that the Wlan0 is configured with DHCP.

3.6 Howto add support for QT5

- Download Meta-QT from the repository
- Add the proper packages to the IMAGE_APPEND variable.

3.7 How to setup a DHCP server on the Raspberry

In the `local.conf`, add:

```
CORE_IMAGE_EXTRA_INSTALL += "dhcp-server "
```

Once the image has booted, edit the file `/etc/default/dhcp-server`: by default it contains:

```
INTERFACES=""
```

The variable INTERFACE must contain the interfaces the DHCP daemon should listen to. Example:

```
# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
#       Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACES="eth0"
```

Next, edit the `/etc/dhcp/dhcp.conf` file. Example:

```
ddns-update-style none;

option domain-name "dragon.org";
option domain-name-servers ns1.example.org, ns2.example.org;

# default lease time is one week.
default-lease-time 604800;

# max-lease-time 30 days
max-lease-time 2592000;

authoritative;

log-facility local7;

subnet 169.254.1.0 netmask 255.255.255.0 {
  range 169.254.1.25 169.254.1.225;
}
```

The leased addresses are contained in the file: `/var/lib/dhcp/dhcp.leases`

3.8 How add Docker to the Raspberry PI image

Tested on 20201027 on Yocto Dunfell

Add the Meta-Virtualization layer.

```
git clone --depth 1 -b dunfell git://git.yoctoproject.org/meta-virtualization
```

Add the following section to the Local.CONF

```
CORE_IMAGE_EXTRA_INSTALL += "kernel-modules docker-ce bash openssh ntp "
DISTRO_FEATURES_append = " virtualization"
IMAGE_ROOTFS_EXTRA_SPACE = "10485760"
NOHDD="1"
NOISO="1"
```

NTP is needed, otherwise the clock is not updated and it is not possible to download images from the docker hub.

The resulting system is still System V, some docs say that System D is needed by Docker, but it is not true.

The BBLAYERS in bblayers.conf is:

```
BBLAYERS ?= " \
  /home/minguzzi/quantavis_yocto_platform/poky/meta \
  /home/minguzzi/quantavis_yocto_platform/poky/meta-poky \
  /home/minguzzi/quantavis_yocto_platform/poky/meta-yocto-bsp \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-oe \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-multimedia \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-networking \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-filesystems \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-perl \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-python \
  /home/minguzzi/quantavis_yocto_platform/meta-openembedded/meta-webserver \
  /home/minguzzi/quantavis_yocto_platform/meta-raspberrypi \
  /home/minguzzi/quantavis_yocto_platform/meta-rbplatform \
  /home/minguzzi/quantavis_yocto_platform/meta-virtualization \
  "
```

3.8.1 Links

- https://jacobncalvert.com/2019/12/22/building-a-customized-linux-image-for-raspberry-pi-with-yocto-docker-support/
  The most useful
- https://wiki.yoctoproject.org/wiki/TipsAndTricks/DockerOnImage

3.9 How add the NTP client to the image

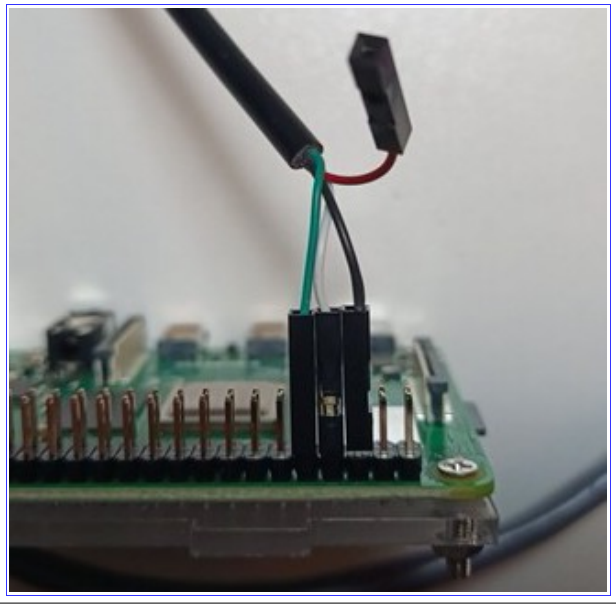
Add the following to the Local.CONF

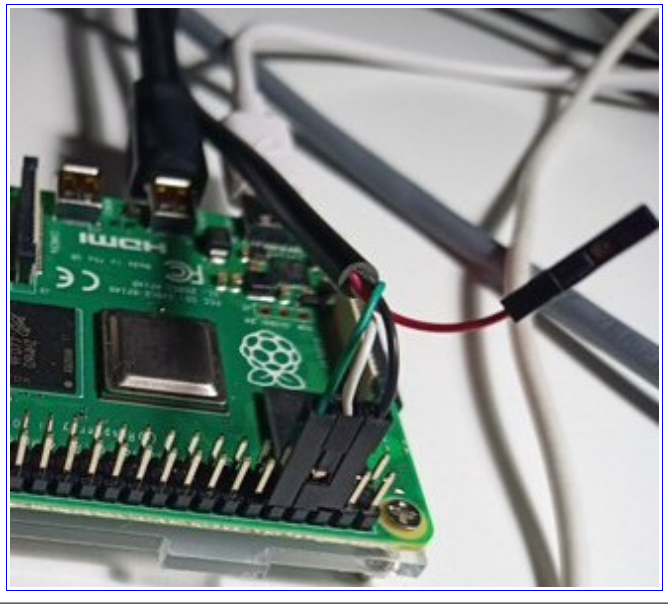
```
CORE_IMAGE_EXTRA_INSTALL += " ntp "
```

Link: Configure the NTP client

3.10 How to connect the serial line to use the console

The USB to serial line can be connected as shown in the image below.





3.10.1 Setting up the Screen serial terminal

The command line tool `screen` was used . Usage example: `sudo screen /dev/ttyUSB0 115200`
Once the tool is running, it is possible to quit by pressing Control-A and then typing `:quit`

3.11 How to set up the NFS (Network File System)

3.11.1 NTF Server on the Linux Yocto

Adds an `/etc/exports` file to configure the exported directories. Example:

```
/var/shared 169.254.1.0/24(rw,sync,no_root_squash)
```

Shares the directory `/var/shared` with the hosts of the subnet `169.254.1.0/24`. The specified options mean: read/write access. no_root_squash is needed, otherwise the client fails to write on the shared folder (permission denied).

3.11.2 NTF Client

On Ubuntu, install the package nfs-common.

```
sudo mount 169.254.1.10:/var/shared /home/minguzzi/wrkdir/yocto/tmpmount/
```

The command:

```
 showmount -e  169.254.1.10
```

Shows the exported directories on a given host

3.12 How to override the files of a Yocto recipe with a custom recipe (Hard-won technique)

The custom recipe is called META-CUSTOM. For example,we want to override the `/etc/network/interfaces` file, that by default is written by the Yocto recipe `init-ifupdown`. The directory tree is

- `meta-custom`
  - `conf`
  - `recipes-core`
    - `init-ifupdown`
      - `init-ifupdown_%.bbappend` (file)
      - `files`
        - `interfaces` (file)

The content of the `init-ifupdown_%.bbappend` is below. The `%` makes sure it will work with all versions of the recipe.

```
FILESEXTRAPATHS_prepend := "${THISDIR}/files:"
```

The content of the `interfaces` file is:

```
# The loopback interface
auto lo
iface lo inet loopback

# Wireless interfaces
iface wlan0 inet dhcp
        wireless_mode managed
        wireless_essid any
        wpa-driver wext
        wpa-conf /etc/wpa_supplicant/wpa_supplicant.conf

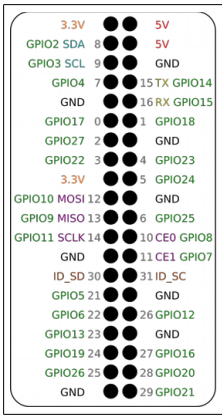
# Wired or wireless interfaces
auto eth0
iface eth0 inet static
        address 169.254.1.10
        netmask 255.255.255.0
        gateway 169.254.1.1
```

4 TODO Next

- NFS
- Acquire images from the camera
- Detect people
- Measure the temperature
- Send images to a connected client, connected by:
  - Wifi
  - Bluetooth

5 Appendices

5.1 GPIO PIN(s)



Raspberry Pi B+ Leaf

Power (5 Volts)
Power (3 Volts)
Ground
WiringPi GPIO
BCM GPIO
I2C Interface
UART Interface
SPI Interface
ID EEPROM Interface

splitbrain.org